JSON

CHEAT SHEETS

A Complete Reference for Data Interchange & APIs

Syntax & Structure · Data Types · Objects & Arrays · JSON Schema · Parsing & Serialization · JSONPath & jq · REST APIs · JWT & GeoJSON · NDJSON & JSON5 · Schema in Practice · Zod & Pydantic · OpenAPI · JSON Patch · Gotchas

15 REFERENCE SHEETS

CONTENTS

FUNDAMENTALS

01 Syntax & Structure
02 Values & Data Types
03 Objects & Arrays
04 JSON Schema

APPLIED

05 Parsing & Serialization
06 JSONPath & Querying
07 JSON in APIs & HTTP
08 Advanced & Related Formats

VALIDATION & APIS

09 JSON Schema in Practice
10 Typed Validation in Code
11 OpenAPI & JSON Bodies
12 JSON Patch & Merge Patch
13 Gotchas & Interop

QUICK REFERENCE

14 Quick Reference — Syntax & Schema
15 Quick Reference — Parsing, Querying & APIs

Covers RFC 8259 · JSON Schema Draft 2020-12 · RFC 9535 JSONPath · jq · JWT · GeoJSON (2025) Begin → Sheet 01

JSON Reference

Table of Contents

15 reference sheets · syntax · examples · notes

Sheet 1
Syntax & Structure
spec overview · key-value pairs · nesting · whitespace · encoding · escape sequences

Sheet 2
Values & Data Types
string · number · boolean · null · array · object · type rules · edge cases

Sheet 3
Objects & Arrays
object syntax · array syntax · nested structures · mixed types · empty values · ordering

Sheet 4
JSON Schema
type validation · required · properties · $ref · $defs · allOf · anyOf · oneOf

Sheet 5
Parsing & Serialization
parse · stringify · pretty-print · error handling · JS · Python · PHP · Ruby · Go

Sheet 6
JSONPath & Querying
JSONPath syntax · dot notation · wildcards · filters · recursive descent · jq

Sheet 7
JSON in APIs & HTTP
REST conventions · Content-Type · request/response shapes · pagination · RFC 7807 errors

Sheet 8
Advanced & Related Formats
JSON5 · JSONC · NDJSON · BSON · JWT · GeoJSON · format selection guide

Sheet 9
JSON Schema in Practice
$schema drafts · if / then / else · dependentRequired · unevaluatedProperties · Ajv · jsonschema

Sheet 10
Typed Validation in Code
z.object · safeParse · z.infer · BaseModel · model_validate_json · ValidationError

Sheet 11
OpenAPI & JSON Bodies
openapi: 3.1 · requestBody · responses · components/schemas · $ref · readOnly · examples

Sheet 12
JSON Patch & Merge Patch
RFC 6902 · op · path · add · remove · replace · move · copy · test · RFC 7396 merge patch

Sheet 13
Gotchas & Interop
Number.MAX_SAFE_INTEGER · BigInt · dates · NaN · undefined · duplicate keys · BOM · key order

Sheet 14
Quick Reference — Syntax & Schema
syntax rules · value types · escapes · JSON Schema types · object & array keywords · composition · formats · common errors

Sheet 15
Quick Reference — Parsing, Querying & APIs
parse & serialize by language · JSON.stringify options · JSONPath · jq · Pointer & Patch · HTTP & APIs · related formats

Syntax & Structure

WHAT IS JSON?

JSON (JavaScript Object Notation) is a lightweight, text-based data-interchange format defined by **RFC 8259** and **ECMA-404**. Despite its name, JSON is completely language-independent and is the dominant format for REST APIs, config files, and data storage. A JSON document is always encoded in **UTF-8** (UTF-16 and UTF-32 are also permitted by the spec but rarely used in practice). Every valid JSON document is either a single *value* — an object, array, string, number, boolean, or null.

KEY RULES

- Keys **must** be double-quoted strings — single quotes are invalid
- Values must be one of the six types: object, array, string, number, boolean, or `null`
- Trailing commas are **not** allowed after the last item in an object or array
- Comments are **not** part of the spec — strip them before parsing
- Whitespace (spaces, tabs, newlines) outside strings is insignificant
- The root value may be any JSON type, not just an object

ANNOTATED JSON DOCUMENT

```
{
  "name": "Alice",
  "age": 30,
  "active": true,
  "score": 9.5,
  "nickname": null,
  "tags": ["admin", "editor"],
  "address": {
    "city": "Austin",
    "zip": "78701"
  }
}
```

LINE NOTES

- → `"name": "Alice"` — string value; key and value both double-quoted
- → `"age": 30` — integer number; no quotes around the value
- → `"active": true` — boolean literal; must be lowercase
- → `"score": 9.5` — floating-point number
- → `"nickname": null` — null literal; must be lowercase
- → `"tags": [...]` — array of strings as a value
- → `"address": {...}` — nested object as a value

DOCUMENT STRUCTURE RULES

RULE	VALID	INVALID
Key quotes	`"key": 1`	`key: 1`
String quotes	`"hello"`	`'hello'`
Trailing comma	`{"a":1}`	`{"a":1,}`
Boolean case	`true` / `false`	`True` / `FALSE`
Null case	`null`	`Null` / `NULL`
Comments	— not supported —	`// comment`
Root type	any JSON value	bare identifier

ENCODING & ESCAPES

ESCAPE	CHARACTER
`\"`	Double quote
`\\`	Backslash
`\/`	Forward slash (optional)
`\n`	Newline
`\r`	Carriage return
`\t`	Tab
`\uXXXX`	Unicode code point (4 hex digits)

COMMON MISTAKES

- Using single quotes for keys or strings — only double quotes are valid
- Leaving a trailing comma after the last object property or array element
- Writing `True`, `False`, or `None` (Python style) instead of lowercase JSON literals
- Embedding comments with `//` or `/* */` — parsers will reject them

TIPS

- Use a linter or validator (jsonlint.com, VS Code) to catch syntax errors before runtime
- Prefer 2-space indentation for readability; minify for production transport
- If you need comments in config files, consider **JSON5** or **JSONC** (see Sheet 08)

SUMMARY

- JSON is defined by RFC 8259 — language-independent, UTF-8 text
- Six value types: object · array · string · number · boolean · null
- Keys must be double-quoted; no trailing commas; no comments
- Booleans and null are lowercase literals: `true`, `false`, `null`

Values & Data Types

THE SIX JSON TYPES

JSON defines exactly **six value types**. There is no distinction between integers and floats — all numbers share a single type. There are no date, undefined, function, or binary types; these must be represented as strings or omitted. Understanding the limits of each type prevents silent data loss, especially with **large integers** and **special float values** like `Infinity` and NaN, which are not valid JSON.

TYPE QUICK RULES

- `string` — always double-quoted; escape special chars with `\`
- `number` — no `NaN`, no `Infinity`, no leading zeros (except `0.x`)
- `boolean` — only `true` or `false` (lowercase)
- `null` — represents absence of a value; distinct from `false` or `0`
- `object` — unordered collection of key/value pairs wrapped in `{ }`
- `array` — ordered list of values wrapped in `[ ]`; may mix types
- JSON has no `undefined` — missing keys and `null` values are different things

ALL SIX TYPES IN ONE DOCUMENT

```
{
  "type_string":  "Hello, World!",
  "type_escape":  "Line 1\nLine 2\t(tab)",
  "type_unicode": "\u00e9l\u00e8ve",

  "type_integer": 42,
  "type_float":   3.14159,
  "type_exp":     2.5e10,
  "type_neg":     -7,

  "type_true":    true,
  "type_false":   false,

  "type_null":    null,

  "type_array":   [1, "two", true, null],

  "type_object":  { "x": 1, "y": 2 }
}
```

NOTES

- → `\n`, `\t` — escape sequences inside strings; raw newlines not allowed
- → `\uXXXX` — any Unicode character by hex code point
- → `2.5e10` — scientific notation is valid; equals 25000000000
- → `-7` — negative numbers use a leading minus; no leading plus allowed
- → Arrays may contain mixed types — no schema enforcement in bare JSON
- → `null` ≠ `false` ≠ `0` ≠ `""` — each is a distinct value

NUMBER RULES & LIMITS

FORM	EXAMPLE	NOTES
Integer	42	No size limit in spec; parsers may differ
Negative	−3	Leading + is invalid
Float	3.14	Must have digit before decimal point
Exponent	1e8	e or E both valid
Leading zero	0.5	07 is invalid
NaN	— invalid —	Not in spec; use `null` instead
Infinity	— invalid —	Not in spec; use `null` or string

STRING RULES & EDGE CASES

CASE	VALID JSON
Empty string	`""`
Whitespace only	`" "`
Unicode	`"\u4e2d\u6587"`
Surrogate pair	`"\uD83D\uDE00"`
Embedded quote	`"say \"hi\""`
Backslash	`"C:\\path"`
Raw newline in string	— invalid; use \n —

COMMON MISTAKES

- Using `NaN` or `Infinity` in number fields — neither is valid JSON
- Storing dates as numbers (Unix timestamps) without documenting the unit (ms vs s)
- Assuming large integers are safe — parsers using IEEE 754 doubles lose precision above 2^{53}
- Treating `null` and a missing key as equivalent — they have different semantics

TIPS

- Represent dates as ISO 8601 strings: `"2024-06-15T14:30:00Z"`
- For large integers (IDs, snowflakes), use strings to avoid precision loss across languages
- Use `null` explicitly to distinguish "value is absent" from "value is empty string"

SUMMARY

- Six types: `string` · `number` · `boolean` · `null` · `object` · `array`
- Numbers: no `NaN`, no `Infinity`, no leading zeros (except `0.x`)
- Strings must be double-quoted; use `\uXXXX` for Unicode escapes
- Arrays are ordered and may mix types; objects are unordered key-value maps

Objects & Arrays

OBJECTS & ARRAYS

A JSON **object** is an *unordered* collection of zero or more key/value pairs enclosed in curly braces `{ }`. Keys must be unique strings within one object (duplicate keys are technically undefined behaviour). A JSON **array** is an *ordered* sequence of zero or more values enclosed in square brackets `[ ]`. Arrays preserve insertion order and may contain any mix of the six JSON types, including other arrays or objects, enabling arbitrarily deep nesting.

KEY RULES

- Object key order is **not guaranteed** — never rely on it for logic
- Duplicate keys in one object: spec says undefined; most parsers keep the last value
- Array elements are accessed by zero-based index, not by key
- Empty object `{}` and empty array `[]` are both valid JSON values
- Arrays and objects can nest to any depth — but keep it manageable for humans
- A single comma separates each element; no comma after the last

OBJECTS, ARRAYS & NESTING

```
{
  "user": {
    "id": 1,
    "name": "Bob",
    "roles": ["admin", "editor"],
    "address": {
      "city": "Denver",
      "coords": [39.7392, -104.9903]
    }
  },

  "empty_obj": {},
  "empty_arr": [],

  "matrix": [
    [1, 2, 3],
    [4, 5, 6]
  ],

  "mixed": [42, "hello", true, null, {"k": "v"}]
}
```

NOTES

- → `"roles": [...]` — array as an object value; common pattern for tags/permissions
- → `"coords": [lat, lng]` — array of numbers; positional meaning is by convention
- → `"empty_obj": {}` — valid; represents an object with no properties yet
- → `"empty_arr": []` — valid; represents a list with zero items
- → `"matrix"` — array of arrays; creates a 2D structure
- → `"mixed"` — arrays can contain any type mix including nested objects

OBJECT PATTERNS

PATTERN	EXAMPLE	NOTES
Simple pair	`{"k": "v"}`	Minimal object
Multi-key	`{"a":1,"b":2}`	Comma-separated, no trailing comma
Nested object	`{"a":{"b":1}}`	Any depth
Array value	`{"tags": ["x","y"]}`	Array as property value
Null value	`{"x": null}`	Key present, value absent
Empty	`{}`	Zero properties

ARRAY PATTERNS

PATTERN	EXAMPLE	NOTES
String list	`["a","b","c"]`	Ordered, indexed from 0
Number list	`[1,2,3]`	Homogeneous is conventional
Mixed types	`[1,"x",null]`	Spec allows; avoid if avoidable
Array of objects	`[{"id":1},{"id":2}]`	Most common API response shape
Nested arrays	`[[1,2],[3,4]]`	Matrix / grid data
Empty	`[]`	Zero elements

COMMON MISTAKES

- Relying on object key order — it is not guaranteed by the spec or all parsers
- Using duplicate keys in the same object — behaviour is undefined and parser-dependent
- Trailing comma after last array element or object property
- Confusing positional array meaning — `[lat, lng]` vs `[lng, lat]` requires documentation

TIPS

- Prefer arrays of objects with named keys over positional arrays for clarity
- Use empty arrays `[]` rather than `null` for absent list values — easier to iterate
- Keep nesting to 3–4 levels maximum; deeper structures are a sign to redesign the schema

SUMMARY

- Objects: unordered key/value pairs in `{ }` — keys are unique strings
- Arrays: ordered values in `[ ]` — zero-indexed, mixed types allowed
- Both can be empty (`{}`, `[]`) and nested to any depth
- Array of objects is the most common shape for list responses in REST APIs

JSON Schema

type validation · required · properties · $ref · $defs · allOf · anyOf · oneOf

WHAT IS JSON SCHEMA?

JSON Schema is a vocabulary (draft standardized at *json-schema.org*) for describing the structure and constraints of JSON documents. A schema is itself a JSON object containing keywords like `type`, `properties`, and `required`. Validators (ajv, jsonschema, etc.) use the schema to check whether a JSON instance conforms. Schemas are used for API documentation, form validation, code generation, and config file enforcement. The current stable release is **Draft 2020-12**.

CORE KEYWORDS

- `$schema` — declares which draft is being used
- `type` — constrains to one or more JSON types
- `properties` — defines allowed keys and their sub-schemas for objects
- `required` — array of keys that must be present
- `$defs` — named reusable sub-schemas (replaces old `definitions`)
- `$ref` — reference to another schema by JSON Pointer URI
- `additionalProperties` — `false` to disallow unlisted keys

FULL SCHEMA EXAMPLE

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://example.com/user.schema.json",
  "title": "User",
  "type": "object",
  "required": ["id", "name", "email"],
  "properties": {
    "id":    { "type": "integer", "minimum": 1 },
    "name":  { "type": "string",  "minLength": 1, "maxLength": 100 },
    "email": { "type": "string",  "format": "email" },
    "age":   { "type": "integer", "minimum": 0, "maximum": 150 },
    "role":  { "$ref": "#/$defs/role" },
    "tags":  { "type": "array", "items": { "type": "string" } }
  },
  "additionalProperties": false,
  "$defs": {
    "role": {
      "type": "string",
      "enum": ["admin", "editor", "viewer"]
    }
  }
}
```

NOTES

- → `$schema` — tells validators which draft rules to apply
- → `$id` — canonical URI for this schema; enables cross-schema `$ref`
- → `"required": [...]` — keys listed here must exist in the instance
- → `"minimum": 1` — number constraint; see table for all numeric keywords
- → `"format": "email"` — semantic hint; validators may or may not enforce it
- → `"$ref": "#/$defs/role"` — reuses the `role` definition inline
- → `"additionalProperties": false` — rejects any key not listed in `properties`

VALIDATION KEYWORDS BY TYPE

TYPE	KEYWORD	MEANING
`string`	`minLength` / `maxLength`	Character count bounds
`string`	`pattern`	Regex the value must match
`string`	`format`	Semantic format hint (email, uri, date…)
`string`	`enum`	Value must be one of the listed options
`number`	`minimum` / `maximum`	Inclusive bounds
`number`	`exclusiveMinimum`	Value must be strictly greater
`number`	`multipleOf`	Value must be a multiple of N
`array`	`items`	Schema applied to every element
`array`	`minItems` / `maxItems`	Array length bounds
`array`	`uniqueItems`	`true` to require distinct elements
`object`	`required`	Array of mandatory key names
`object`	`additionalProperties`	`false` = no extra keys allowed

COMPOSITION KEYWORDS

KEYWORD	MEANING
`allOf`	Must match ALL listed schemas (AND)
`anyOf`	Must match AT LEAST ONE schema (OR)
`oneOf`	Must match EXACTLY ONE schema (XOR)
`not`	Must NOT match the given schema
`if / then / else`	Conditional schema application
`$ref`	Reuse any schema by URI or JSON Pointer
`$defs`	Named sub-schema store within a document

COMMON MISTAKES

- Confusing `anyOf` and `oneOf` — use `oneOf` only when exactly one branch should match
- Omitting `additionalProperties: false` then being surprised extra keys are accepted
- Using old `definitions` key — it was replaced by `$defs` in Draft 2019-09
- Assuming `format` is enforced — most validators treat it as advisory by default

TIPS

- Use `$defs` to DRY up repeated sub-schemas and reference with `$ref`
- Set `"additionalProperties": false` on API payloads to catch typos in field names
- Test schemas with jsonschema.dev or ajv-cli before deploying

SUMMARY

- JSON Schema validates JSON structure using keyword-based rules
- `type` · `properties` · `required` are the three most-used keywords
- `$defs` + `$ref` enable reusable, composable schemas
- `allOf` / `anyOf` / `oneOf` combine schemas with AND / OR / XOR logic

Parsing & Serialization

parse · stringify · pretty-print · error handling · language comparison

PARSE VS SERIALIZE

Parsing (deserializing) converts a JSON string into a native data structure — a dictionary, map, object, or hash depending on the language. **Serialization** (stringifying / marshalling) is the reverse: converting native data into a JSON string. Every major language ships a JSON library in its standard library. The table below shows the canonical function names; all do the same thing under different names.

UNIVERSAL CONCEPTS

- **Parse** — JSON string → native object/dict/map
- **Stringify** — native value → JSON string
- **Pretty-print** — serialize with indentation for readability
- Always wrap parse calls in error handling — malformed JSON throws
- Circular references in native objects cannot be serialized to JSON
- `undefined` (JS), functions, and binary data are stripped on serialize

PARSE & SERIALIZE — FOUR LANGUAGES

```
// ── JavaScript ──────────────────────────────────────
const obj  = JSON.parse('{"name":"Alice","age":30}');
const str  = JSON.stringify(obj);              // compact
const pretty = JSON.stringify(obj, null, 2);   // indented

// ── Python ──────────────────────────────────────────
# import json
obj    = json.loads('{"name":"Alice","age":30}')
str    = json.dumps(obj)                       # compact
pretty = json.dumps(obj, indent=2)             # indented

// ── PHP ─────────────────────────────────────────────
$obj  = json_decode('{"name":"Alice","age":30}', true);
$str  = json_encode($obj);                     // compact
$pretty = json_encode($obj, JSON_PRETTY_PRINT); // indented

// ── Ruby ────────────────────────────────────────────
# require 'json'
obj    = JSON.parse('{"name":"Alice","age":30}')
str    = obj.to_json                           # compact
pretty = JSON.pretty_generate(obj)             # indented
```

NOTES

- → JS `JSON.parse` → throws `SyntaxError` on bad input; wrap in try/catch
- → JS `JSON.stringify(obj, null, 2)` — second arg is a replacer, third is indent spaces
- → Python `json.loads` → raises `json.JSONDecodeError` on bad input
- → Python `json.dumps(obj, indent=2)` also accepts `sort_keys=True` for stable output
- → PHP `json_decode(..., true)` — second arg `true` returns assoc array, not stdClass
- → Ruby `JSON.parse` → raises `JSON::ParserError`; rescue it explicitly

PARSE & STRINGIFY BY LANGUAGE

LANGUAGE	PARSE (STRING → NATIVE)	STRINGIFY (NATIVE → STRING)
JavaScript	JSON.parse(str)	JSON.stringify(val)
Python	json.loads(str)	json.dumps(val)
PHP	json_decode(str, true)	json_encode(val)
Ruby	JSON.parse(str)	val.to_json
Java	gson.fromJson(str, T)	gson.toJson(val)
Go	json.Unmarshal([]byte, &v)	json.Marshal(v)
Rust	serde_json::from_str(str)	serde_json::to_string(val)
C#	JsonSerializer.Deserialize<T>(str)	JsonSerializer.Serialize(val)

ERROR TYPES BY LANGUAGE

LANGUAGE	PARSE ERROR TYPE
JavaScript	SyntaxError
Python	json.JSONDecodeError
PHP	returns null; check json_last_error()
Ruby	JSON::ParserError
Java (Gson)	JsonSyntaxException
Go	*json.SyntaxError
Rust	serde_json::Error
C#	JsonException

COMMON MISTAKES

- Not catching parse errors — malformed input from an API will crash unguarded code
- PHP: forgetting the `true` second arg to `json_decode` and getting a stdClass instead of array
- Serializing objects with circular references — all languages will error or loop infinitely
- Using pretty-print in production API responses unnecessarily — adds bytes and latency

TIPS

- Always validate + parse in one step — never manipulate raw JSON strings with regex
- Use `sort_keys=True` (Python) or replacer sort in JS for deterministic, diffable output
- For streaming large files, prefer streaming parsers (Python `ijson`, Node.js `stream-json`)

SUMMARY

- Parse = JSON string → native; Stringify = native → JSON string
- Every major language has a built-in JSON library — no third-party package needed for basics
- Always handle parse errors — invalid JSON throws or returns null depending on language
- Pretty-print with indent option for debugging; compact for production transport

JSONPath & Querying

JSONPATH

JSONPath (RFC 9535, 2024) is a query language for extracting values from JSON documents, analogous to XPath for XML. An expression always starts with $ (the root node) and chains selectors with either **dot notation** (`$.store.book`) or **bracket notation** (`$["store"]["book"]`). Both forms are equivalent. JSONPath is supported natively in PostgreSQL, MySQL, AWS Step Functions, many API gateways, and libraries for every major language.

SYNTAX QUICK REFERENCE

- `$` — root of the document
- `.key` — child property by name
- `[n]` — array element at index n (zero-based)
- `[*]` or `.*` — all children (wildcard)
- `..` — recursive descent (all descendants)
- `[?(@.price < 10)]` — filter expression
- `[1:3]` — array slice (elements 1 and 2)

JQ — COMMAND-LINE JSON PROCESSOR

```
# ── Basic access ──────────────────────────────────────
echo '{"name":"Alice","age":30}' | jq '.name'        # "Alice"
echo '{"name":"Alice","age":30}' | jq '.age'         # 30

# ── Arrays ────────────────────────────────────────────
echo '[1,2,3]' | jq '.[0]'                           # 1
echo '[1,2,3]' | jq '.[]'                            # 1  2  3  (each on own line)
echo '[1,2,3]' | jq '.[1:3]'                         # [2,3]

# ── Filters & pipes ───────────────────────────────────
cat data.json | jq '.users[] | .name'                # each user's name
cat data.json | jq '.users[] | select(.age > 25)'    # filter by condition
cat data.json | jq '[.users[] | .name]'              # collect into array

# ── Transform ─────────────────────────────────────────
cat data.json | jq '{id: .id, label: .name}'         # reshape object
cat data.json | jq '.users | length'                 # count items
cat data.json | jq '.users | sort_by(.age)'          # sort array
cat data.json | jq '.users | map(.age) | add'        # sum all ages

# ── Output options ────────────────────────────────────
jq -r '.name'       # raw output (no quotes around strings)
jq -c '.'           # compact (minified) output
jq '.' file.json    # pretty-print a file
```

NOTES

- `.` — identity filter; outputs the input unchanged
- `.key` — accesses object property; `.a.b` chains
- `.[]` — iterates every element of an array or object
- `|` — pipe: output of left becomes input of right
- `select(cond)` — keeps input only if condition is true
- `[expr]` — collect filter output into an array
- `-r` flag — strips surrounding quotes from string output

JSONPATH EXPRESSION REFERENCE

EXPRESSION	SELECTS
`$`	Root node
`$.store`	Child `store` of root
`$.store.book[0]`	First book
`$.store.book[*]`	All books
`$..author`	All `author` values anywhere
`$.store.book[-1]`	Last book (RFC 9535)
`$.store.book[0,1]`	First and second book
`$.store.book[?(@.price<10)]`	Books cheaper than 10
`$.store.book[?(@.isbn)]`	Books that have an isbn key
`$..price`	All price values in document

JQ BUILT-IN FUNCTIONS

FUNCTION	PURPOSE
`length`	Count elements or string chars
`keys`	Array of object keys, sorted
`values`	Array of object values
`has("k")`	True if key exists
`map(f)`	Apply filter to each element
`select(f)`	Keep input if filter is true
`sort_by(.k)`	Sort array by key
`group_by(.k)`	Group array by key value
`unique_by(.k)`	Deduplicate by key
`add`	Sum / concatenate array elements
`to_entries`	Object → [{key,value}] array
`from_entries`	[{key,value}] → object

COMMON MISTAKES

- Forgetting `$` at the start of a JSONPath expression — it's mandatory
- jq: omitting `-r` when piping string output to shell — the extra quotes break commands
- Mixing JSONPath implementations — filter syntax varies between tools (Goessner vs RFC 9535)
- Using `..` (recursive descent) on large documents — it can be slow on deep structures

TIPS

- Test JSONPath expressions at jsonpath.com before embedding in code
- Use `jq -c` to produce compact JSON for piping into other tools or curl
- In jq, wrap multi-step transforms in a function with `def name: expr;` to keep scripts readable

SUMMARY

- JSONPath: `$` is root · `.` is child · `[*]` is wildcard · `..` is recursive
- Filter expressions `[?(@.field op value)]` select matching array elements
- jq is the standard CLI tool — pipe-based, filter-oriented, and very fast
- jq's `map`, `select`, `sort_by`, and `group_by` cover most data-shaping tasks

JSON in APIs & HTTP

REST conventions · headers · request/response · pagination · error patterns

JSON AS THE API WIRE FORMAT

JSON is the *de-facto* wire format for REST and most HTTP APIs. The client signals it is sending JSON with `Content-Type: application/json` and requests JSON back with `Accept: application/json`. Well-designed APIs return consistent envelope shapes — a data wrapper for success responses and a structured error object for failures — so clients can handle both cases with the same parsing logic.

HTTP HEADER RULES

- `Content-Type: application/json` — set on any request body you send
- `Accept: application/json` — tells the server what format you want back
- `Content-Type: application/json; charset=utf-8` — explicit encoding (usually redundant)
- GET requests never have a body — pass data in query params, not JSON
- 201 Created should return the newly created resource in the body
- 204 No Content returns an empty body — don't try to parse it

COMMON API JSON SHAPES

```
// ── Success: single resource ──────────────────────
{
  "data": { "id": 1, "name": "Alice", "email": "alice@example.com" }
}

// ── Success: collection with pagination ───────────
{
  "data": [
    { "id": 1, "name": "Alice" },
    { "id": 2, "name": "Bob" }
  ],
  "meta": {
    "total": 200,
    "page": 1,
    "per_page": 20,
    "next": "/users?page=2"
  }
}

// ── Error response (RFC 7807 Problem Details) ─────
{
  "type":   "https://example.com/errors/validation",
  "title":  "Validation Failed",
  "status": 422,
  "detail": "The 'email' field is not a valid address.",
  "errors": [
    { "field": "email", "message": "Invalid format" }
  ]
}
```

NOTES

- → `"data"` envelope — wrapping the payload lets you add metadata later without breaking clients
- → `"meta"` block — pagination, counts, and cursor values live here, not inside `data`
- → `"next"` cursor — cursor-based pagination is safer than page numbers for live data
- → RFC 7807 error shape — `type`, `title`, `status`, `detail` are the standard fields
- → `"errors"` array — per-field validation errors alongside the top-level error message
- → Always return JSON error bodies even on 4xx/5xx — never let plain-text stack traces leak

HTTP METHODS & JSON BODY RULES

METHOD	HAS BODY?	TYPICAL JSON USE
GET	No	Returns resource or collection in response
POST	Yes	Create; send new resource data in body
PUT	Yes	Full replace; send complete resource
PATCH	Yes	Partial update; send only changed fields
DELETE	Rarely	Usually no body; 204 or deleted resource back

COMMON STATUS CODES & JSON BODY

CODE	MEANING	BODY
200	OK	Requested data
201	Created	New resource
204	No Content	Empty — do not parse
400	Bad Request	Error + validation details
401	Unauthorized	Auth error message
403	Forbidden	Permission error
404	Not Found	Error message
422	Unprocessable	Validation errors array
429	Rate Limited	Retry-After info
500	Server Error	Generic error — no internals

COMMON MISTAKES

- Returning a 200 status with an error message in the body — use the correct 4xx/5xx code
- Forgetting `Content-Type: application/json` on POST/PUT — server may reject or misparse the body
- Trying to parse a 204 No Content response body — it is empty by definition
- Exposing stack traces or internal error messages in 500 responses

TIPS

- Always check the HTTP status code before parsing the body — don't assume success
- Adopt RFC 7807 Problem Details for error responses — it's becoming the industry standard
- Use cursor-based pagination (`"next_cursor"`) instead of page numbers for stable, live data sets

SUMMARY

- Set `Content-Type: application/json` on every request with a body
- Wrap responses in a `data` envelope to allow adding `meta` later
- Use RFC 7807 for consistent, machine-readable error responses
- Check HTTP status code first — never trust a 200 blindly

Advanced & Related Formats

THE JSON ECOSYSTEM

JSON's strict syntax makes it reliable but sometimes inconvenient — no comments, no trailing commas, no binary data. The ecosystem has produced several **supersets and relatives** that solve specific pain points. **JSON5** adds comments and unquoted keys for config files. **NDJSON** enables streaming of record sequences. **BSON** adds binary types for database storage. **JWT** uses Base64-encoded JSON for authentication tokens. Understanding which format to reach for — and why — is as important as knowing JSON itself.

FORMAT SELECTION GUIDE

- Config files with comments → **JSON5** or **JSONC**
- Streaming / log lines → **NDJSON** / **JSON Lines**
- Database storage with binary data → **BSON** (MongoDB)
- Auth tokens in HTTP headers → **JWT**
- Geographic features → **GeoJSON** (RFC 7946)
- REST API payloads → plain **JSON** (RFC 8259)
- High-performance binary APIs → **MessagePack** or **CBOR**

FORMAT EXAMPLES

```
// ── JSON5 (config file) ──────────────────────────────
{
  // Comments are allowed
  name: 'Alice',            // unquoted keys
  score: +9.5,              // leading + allowed
  hex: 0xFF,                // hex literals
  multiline: "line 1\
line 2",                    // backslash line continuation
  trailing: "ok",           // trailing comma OK
}

// ── NDJSON / JSON Lines (one object per line) ────────
{"id":1,"event":"login","ts":"2024-01-01T00:00:00Z"}
{"id":2,"event":"view","ts":"2024-01-01T00:01:00Z"}
{"id":3,"event":"logout","ts":"2024-01-01T00:05:00Z"}

// ── JWT structure (header.payload.signature) ─────────
// Header  (Base64url-decoded):
{"alg":"HS256","typ":"JWT"}
// Payload (Base64url-decoded):
{"sub":"1234567890","name":"Alice","iat":1516239022}
// Full token:
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiIxMjM0In0.SflKxwR...

// ── GeoJSON (RFC 7946) ───────────────────────────────
{
  "type": "Feature",
  "geometry": {
    "type": "Point",
    "coordinates": [-104.9903, 39.7392]
  },
  "properties": { "name": "Denver" }
}
```

NOTES

- → JSON5 is a superset — all valid JSON is valid JSON5, but not vice versa
- → NDJSON: newline-delimited — stream parsers read one line at a time, no full-document parse needed
- → JWT payload is Base64url encoded, *not* encrypted — never put secrets in it without encryption
- → GeoJSON coordinates are always `[longitude, latitude]` — longitude first (common gotcha)
- → BSON adds types: Date, ObjectId, Binary, Decimal128 — only used with BSON-aware drivers (MongoDB)
- → JSONC (JSON with Comments) is used by VS Code `settings.json` and TypeScript `tsconfig.json`

JSON-RELATED FORMATS COMPARISON

FORMAT	SUPERSET?	COMMENTS	PRIMARY USE
JSON	—	✖	APIs, data interchange
JSON5	✔	✔	Human-written config files
JSONC	✔	✔	IDE configs (VS Code, TS)
NDJSON	✔	✖	Log streaming, bulk import
JSON Lines	✔	✖	Same as NDJSON (alias)
BSON	No (binary)	✖	MongoDB wire protocol
JWT	No (encoded)	✖	Auth tokens in HTTP headers
GeoJSON	✔	✖	Geographic data (RFC 7946)

GEOJSON GEOMETRY TYPES

TYPE	COORDINATES SHAPE
Point	`[lng, lat]`
MultiPoint	`[[lng,lat], ...]`
LineString	`[[lng,lat], [lng,lat], ...]`
Polygon	`[[[lng,lat],...]]` — first/last same
MultiPolygon	Array of Polygon coord arrays
Feature	geometry + arbitrary `properties`
FeatureCollection	Array of Feature objects

COMMON MISTAKES

- Treating JWT as encrypted — the payload is only Base64-encoded and is publicly readable
- GeoJSON: using `[lat, lng]` order — the spec requires `[longitude, latitude]`
- Feeding NDJSON to a standard JSON parser — it will fail; parse line-by-line instead
- Using JSON5 in a production API — most HTTP clients and servers expect strict JSON

TIPS

- Use NDJSON for log ingestion pipelines — append a line per event, no file rewriting needed
- Inspect JWT payloads at jwt.io — paste the token to decode and verify
- For config files that developers edit directly, JSON5 or JSONC is friendlier than strict JSON

SUMMARY

- JSON5 / JSONC add comments & trailing commas — good for config, not APIs
- NDJSON: one JSON object per line — built for streaming and log pipelines
- JWT: Base64url-encoded JSON header + payload — not encrypted by default
- GeoJSON: standard geographic format — coordinates are always `[lng, lat]`

JSON Schema in Practice

$schema drafts · if / then / else · dependentRequired · unevaluatedProperties · Ajv · jsonschema

BEYOND THE CORE KEYWORDS

Sheet 04 covers `type`, `properties` and `required`. Real payloads also need **conditional rules** ("if it's a card payment, a card number is required"), **dependencies between fields**, and a way to reject unknown keys that still works when schemas are combined with `allOf`. This sheet covers those keywords, the differences between drafts that break schemas copied from old tutorials, and how to run validation in JavaScript (**Ajv**) and Python (**jsonschema**).

WHICH DRAFT?

- `$schema` decides the rules; validators default to different drafts if it is missing
- **draft-07** — still the most common in tutorials and older tools
- **2019-09** — added `$defs`, `dependentRequired`, `unevaluatedProperties`
- **2020-12** — current; `prefixItems` replaced the array form of `items`
- OpenAPI 3.1 uses 2020-12; OpenAPI 3.0 uses a modified draft-04/05 subset

NOTES

- → `"if"` — a subschema; when the data matches it, `"then"` applies, otherwise `"else"`
- → `"required": ["method"]` inside `if` — without it, a payload with no `method` matches `if` and gets the card rule
- → `"dependentRequired"` — if `billing` is present, `zip` must be too
- → `"unevaluatedProperties": false` — rejects keys no subschema (including `allOf` / `if` branches) looked at
- → `Ajv2020` — the default `ajv` import only understands draft-07; 2020-12 schemas need the `ajv/dist/2020` build

CONDITIONAL PAYMENT SCHEMA (2020-12)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "properties": {
    "method":  { "enum": ["card", "bank"] },
    "card":    { "type": "string", "pattern": "^[0-9]{16}$" },
    "iban":    { "type": "string" },
    "billing": { "type": "string" },
    "zip":     { "type": "string" }
  },
  "required": ["method"],
  "if":   { "properties": { "method": { "const": "card" } },
            "required": ["method"] },
  "then": { "required": ["card"] },
  "else": { "required": ["iban"] },
  "dependentRequired": { "billing": ["zip"] },
  "unevaluatedProperties": false
}
```

VALIDATING IT

```
// JavaScript — Ajv (npm i ajv ajv-formats)
import Ajv2020 from "ajv/dist/2020";       // 2020-12 build
import addFormats from "ajv-formats";
const ajv = new Ajv2020({ allErrors: true });
addFormats(ajv);                           // "format": "email" etc.
const validate = ajv.compile(schema);      // compile once, reuse
validate({ method: "card" });              // false
validate.errors[0].message;  // "must have required property 'card'"
```

```
# Python — jsonschema (pip install jsonschema)
from jsonschema import Draft202012Validator
v = Draft202012Validator(schema)
for err in v.iter_errors({"method": "bank", "billing": "x"}):
    print(err.message)    # 'iban' is a required property
                          # 'zip' is a dependency of 'billing'
```

DRAFT CHANGES THAT BREAK OLD SCHEMAS

OLD (DRAFT-07)	NOW (2020-12)	NOTE
`definitions`	`$defs`	old name still works in most validators
`dependencies`	`dependentRequired` / `dependentSchemas`	split in 2019-09
`items: [a, b]`	`prefixItems: [a, b]`	tuple validation
`additionalItems`	`items` (after `prefixItems`)	schema for the rest
`$ref + siblings`	siblings now apply	draft-07 ignored keys next to `$ref`
`format`	annotation only	opt in to enforce (`ajv-formats`)

CONDITIONAL & DEPENDENCY KEYWORDS

KEYWORD	MEANING
`if / then / else`	apply `then` when data matches `if`, else `else`
`dependentRequired`	key A present → keys B, C required
`dependentSchemas`	key A present → whole subschema applies
`const`	value must equal this exactly
`unevaluatedProperties`	rule for keys no subschema evaluated
`contains / minContains`	array must hold N items matching a schema

COMMON MISTAKES

- `additionalProperties: false` inside an `allOf` branch rejects keys defined in the *other* branch — use `unevaluatedProperties: false` at the top level instead
- Loading a 2020-12 schema with the default Ajv import — it fails with "no schema with key or ref"; import `ajv/dist/2020`
- Writing `if` without `required` — a missing field matches every `properties` check, so the `then` branch fires unexpectedly
- Calling `ajv.compile()` on every request — compiling is slow; compile once at startup and reuse the function

TIPS

- Always set `$schema` so every tool reads the schema with the same draft rules
- Use `allErrors: true` for form feedback; leave it off for fast API rejection
- Validate in CI with `ajv validate --spec=draft2020 -s schema.json -d "data/*.json"` or `check-jsonschema --schemafile`
- Validators: Ajv (JS), `jsonschema` (Python), networknt (Java), `santhosh-tekuri/jsonschema` (Go), `JsonSchema.Net` (.NET)

SUMMARY

- `if / then / else` — conditional rules on the same object
- `dependentRequired` — one key requires others
- `unevaluatedProperties` — strict objects that survive `allOf`
- `$schema` — pin the draft; 2020-12 is current
- `Ajv2020` / `Draft202012Validator` — compile once, collect errors

Typed Validation in Code

PARSE, DON'T JUST CAST

`JSON.parse()` returns `any` and `json.loads()` returns a plain `dict` — the type annotations you put on them are promises, not checks. A **schema library** defines the shape once and gives you both a **runtime check** at the boundary (HTTP body, file, queue message) and a **static type** for the rest of the code. **Zod** is the common choice in TypeScript; **Pydantic** (v2) in Python. Both coerce, apply defaults, and return readable, path-based errors.

WHERE TO VALIDATE

- At every trust boundary: request bodies, third-party API responses, config files, queue messages
- Unknown keys: Zod strips them, Pydantic ignores them — both can be made strict
- Coercion: Pydantic converts `"1"` → `1` by default (lax mode); Zod never coerces unless you use `z.coerce`

NOTES

- → `z.object({...})` — schema object; `z.infer<typeof X>` turns it into a TypeScript type
- → `.safeParse()` — returns `{ success, data | error }`; never throws
- → `.parse()` — returns the typed value or throws `ZodError`
- → `BaseModel` — field annotations are the schema; instantiation validates
- → `model_validate_json()` — parses bytes/str directly; faster than `json.loads` then validate
- → `e.errors()` — list of dicts with `loc`, `msg`, `type`, `input`

TYPESCRIPT — ZOD

```ts
import { z } from "zod";

const User = z.object({
  id:    z.number().int().positive(),
  email: z.string().email(),
  role:  z.enum(["admin", "viewer"]).default("viewer"),
  tags:  z.array(z.string()).optional(),
});
type User = z.infer<typeof User>;  // { id: number; email: string; ... }

const res = User.safeParse(JSON.parse(body));
if (!res.success) {
  res.error.issues;   // [{ path: ["email"], message: "Invalid email", ... }]
} else {
  res.data.role;      // "viewer"  (default applied)
}
User.parse(data);     // returns User or throws ZodError
```

PYTHON — PYDANTIC V2

```python
from typing import Literal
from pydantic import BaseModel, ConfigDict, EmailStr, Field, ValidationError

class User(BaseModel):
    model_config = ConfigDict(extra="forbid")      # reject unknown keys
    id: int = Field(gt=0)
    email: EmailStr                                # pip install "pydantic[email]"
    role: Literal["admin", "viewer"] = "viewer"
    tags: list[str] | None = None

u = User.model_validate_json(body)    # parse + validate in one step
u.model_dump_json()                   # back to a JSON string
User.model_json_schema()              # dict in JSON Schema format

try:
    User.model_validate({"id": "x", "email": "a@b.io"})
except ValidationError as e:
    e.errors()[0]["loc"]              # ('id',)
```

SAME RULE IN ZOD AND PYDANTIC

RULE	ZOD	PYDANTIC
string	`z.string()`	`str`
integer > 0	`z.number().int().positive()`	`int = Field(gt=0)`
optional	`.optional()`	`X | None = None`
default	`.default("viewer")`	`= "viewer"`
enum	`z.enum(["a","b"])`	`Literal["a","b"]`
array of strings	`z.array(z.string())`	`list[str]`
nullable	`.nullable()`	`X | None`
custom check	`.refine(fn, msg)`	`@field_validator`

GENERATE INSTEAD OF HAND-WRITING

TOOL	DOES
`z.toJSONSchema()`	Zod 4 → JSON Schema (Zod 3: `zod-to-json-schema`)
`model_json_schema()`	Pydantic model → JSON Schema
`json-schema-to-typescript`	JSON Schema → TS interfaces (`json2ts`)
`datamodel-codegen`	JSON Schema / OpenAPI → Pydantic models
`quicktype`	sample JSON → types in 20+ languages

COMMON MISTAKES

- `const user = JSON.parse(body) as User` — a cast checks nothing; a missing field crashes later, far from the cause
- Validating deep inside business logic instead of at the boundary — bad data has already spread
- Forgetting Pydantic's lax mode — `"42"` silently becomes `42`; use `strict=True` (model or field) when type matters
- Expecting Zod to keep unknown keys — `z.object` strips them by default; use `.passthrough()` (v3) / `z.looseObject()` (v4) to keep

TIPS

- Use `safeParse` in request handlers and turn `issues` into a 400/422 response
- Keep the schema as the single source of truth and derive types (`z.infer`) — never maintain both by hand
- Pydantic's `model_validate_json()` skips building an intermediate `dict` — use it for large bodies
- Use `z.coerce.number()` for query strings and form fields, which always arrive as text

SUMMARY

- `z.object` + `z.infer` — one definition, runtime check + TS type
- `safeParse` / `parse` — result object vs. throw
- `BaseModel` — Python class that validates on creation
- `model_validate_json` / `model_dump_json` — JSON in and out
- `extra="forbid"` / `.strict()` — reject unknown keys

OpenAPI & JSON Bodies

DESCRIBING JSON APIS

OpenAPI is a YAML or JSON document that describes an HTTP API: paths, parameters, and the JSON **request and response bodies**. Bodies are written in JSON Schema, usually defined once under `components/schemas` and reused with `$ref`. From one spec you get interactive docs, generated client SDKs and server stubs, mock servers, and request validation. **OpenAPI 3.1** uses standard JSON Schema 2020-12; **3.0** used its own subset, which is why `nullable: true` still appears everywhere.

KEY IDEAS

- One spec per API, usually `openapi.yaml` in the repo
- Bodies are keyed by media type: `content: application/json: schema:`
- Each response is keyed by status code; `default` catches the rest
- `readOnly` fields (like `id`) appear in responses only; `writeOnly` (like `password`) in requests only
- Errors: reuse one `Problem` schema (RFC 9457 problem details, `application/problem+json`)

NOTES

→ `openapi: 3.1.0` — spec version (not your API version, which goes in `info.version`)

→ `requestBody.required: true` — body must be present; separate from the schema's own `required` list

→ `"201"` — quote status codes in YAML so they stay strings

→ `$ref: "#/components/schemas/User"` — a JSON Pointer into this same document

→ `readOnly: true` — clients should not send `id`; servers return it

→ `type: [string, "null"]` — 3.1 way to allow null; 3.0 wrote `type: string` + `nullable: true`

POST /USERS WITH JSON BODIES (3.1)

```yaml
openapi: 3.1.0
info: { title: Users API, version: 1.0.0 }
paths:
  /users:
    post:
      summary: Create a user
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: "#/components/schemas/User" }
            example: { email: ada@example.com, role: admin }
      responses:
        "201":
          description: Created
          content:
            application/json:
              schema: { $ref: "#/components/schemas/User" }
components:
  schemas:
    User:
      type: object
      required: [email]
      properties:
        id:       { type: integer, readOnly: true }
        email:    { type: string, format: email }
        role:     { type: string, enum: [admin, viewer], default: viewer }
        nickname: { type: [string, "null"] }      # 3.1 nullable
```

3.0 VS 3.1 DIFFERENCES

3.0	3.1
`nullable: true`	`type: [string, "null"]`
`exclusiveMinimum: true` + `minimum`	`exclusiveMinimum: 5` (a number)
`example:` inside schemas	`examples: [...]` (array, JSON Schema style)
siblings of `$ref` ignored	siblings of `$ref` allowed
`format: binary` for uploads	`contentMediaType` / `contentEncoding`
no webhooks	top-level `webhooks:`

TOOLING AROUND A SPEC

TOOL	USE
Swagger UI / Redoc	interactive or static API docs
Spectral	lint the spec (`spectral lint openapi.yaml`)
openapi-generator	client SDKs and server stubs, 50+ languages
openapi-typescript	TS types for request/response bodies
Prism (Stoplight)	mock server from the examples
Schemathesis	property-based tests against a running API

COMMON MISTAKES

- Mixing 3.0 and 3.1 syntax — `nullable: true` is ignored by strict 3.1 tools; pick one version per spec
- Leaving status codes unquoted — `201:` becomes an integer key, which some parsers and linters reject
- Copying the same object into every endpoint — changes drift; define it once in `components/schemas` and `$ref` it
- Marking `id` as `required` on the create request — use `readOnly: true` so it is required only in responses

TIPS

- Lint in CI with `spectral lint` — catches missing descriptions, bad `$ref`s and inconsistent naming
- Add realistic `example` values — docs, mock servers and generated tests all use them
- Generate client types (`openapi-typescript`, `openapi-generator`) rather than hand-writing request shapes
- Validate real requests against the spec in tests so docs and code can't drift apart

SUMMARY

- `requestBody` / `responses` — bodies keyed by media type and status
- `components/schemas` + `$ref` — define once, reuse everywhere
- `readOnly` / `writeOnly` — one schema for request and response
- `openapi: 3.1` — real JSON Schema 2020-12; `type: [x, "null"]`
- `application/problem+json` — standard error body

JSON Patch & Merge Patch

RFC 6902 · op · path · add · remove · replace · move · copy · test · RFC 7396 merge patch

SENDING ONLY THE CHANGES

A PATCH request updates part of a resource. JSON has two standard formats for that. **JSON Patch** (RFC 6902) is an ordered **list of operations** — add, remove, replace, move, copy, test — each aimed at a **JSON Pointer** path. **JSON Merge Patch** (RFC 7396) is a **partial document**: send the keys to change, `null` to delete. Merge Patch is simpler to write; JSON Patch can edit arrays element by element and can fail safely with `test`.

JSON POINTER PATHS (RFC 6901)

- `""` — the whole document; `"/name"` — top-level key
- `"/tags/0"` — array index (zero-based); `"/tags/-"` — one past the end (append)
- `~1` encodes `/` and `~0` encodes `~` in a key: key `"a/b"` → `"/a~1b"`
- Pointers are exact — no wildcards or filters (that is JSONPath, Sheet 06)

ORIGINAL DOCUMENT

```
{ "name": "Ada", "role": "viewer", "tags": ["a", "b"],
  "address": { "city": "Paris", "zip": "75001" } }
```

JSON PATCH — APPLICATION/JSON-PATCH+JSON

```
[
  { "op": "test",    "path": "/role",    "value": "viewer" },
  { "op": "replace", "path": "/role",    "value": "admin" },
  { "op": "add",     "path": "/tags/-",  "value": "c" },
  { "op": "add",     "path": "/tags/0",  "value": "z" },
  { "op": "remove",  "path": "/address/zip" },
  { "op": "copy",  "from": "/name", "path": "/nickname" },
  { "op": "move",  "from": "/address/city", "path": "/city" }
]
// → { "name": "Ada", "role": "admin",
//     "tags": ["z","a","b","c"], "address": {},
//     "nickname": "Ada", "city": "Paris" }
```

MERGE PATCH — APPLICATION/MERGE-PATCH+JSON

```
{ "role": "admin", "address": { "zip": null },
  "tags": ["c"] }
// → { "name": "Ada", "role": "admin", "tags": ["c"],
//     "address": { "city": "Paris" } }
```

NOTES

- → `test` — aborts the **whole** patch if the value differs; use it for optimistic concurrency
- → `add` on an array index inserts and shifts later items; on an object key it creates or overwrites
- → `replace` — like `remove` + `add`, but the target must already exist
- → `move` / `copy` — take a `from` pointer instead of a `value`
- → Operations run in order; if any fails, none are applied
- → Merge Patch: objects merge recursively, `null` deletes, **arrays are replaced whole**
- → Merge Patch cannot set a value to `null` — `null` always means "delete"

APPLYING PATCHES IN JAVASCRIPT

```
import { applyPatch, compare } from "fast-json-patch";
applyPatch(doc, ops).newDocument;  // apply
compare(before, after);            // diff → ops
```

JSON PATCH OPERATIONS

OP	NEEDS	FAILS WHEN
add	path, value	parent missing
remove	path	target missing
replace	path, value	target missing
move	from, path	from missing or is a parent of path
copy	from, path	from missing
test	path, value	value not equal (deep compare)

PATCH VS MERGE PATCH

	JSON PATCH	MERGE PATCH
Content-Type	application/json-patch+json	application/merge-patch+json
Format	array of ops	partial document
Delete a key	remove op	set it to null
Edit one array item	✔ by index	✖ resend whole array
Set a value to null	✔	✖
Conditional	✔ test op	✖ (use If-Match ETag)
Best for	precise edits, arrays	simple forms, settings

LIBRARIES

LANGUAGE	LIBRARY
JavaScript	fast-json-patch, json-merge-patch
Python	jsonpatch, json-merge-patch
Java	zjsonpatch, Jakarta JSON-P
Go	evanphx/json-patch
.NET	Microsoft.AspNetCore.JsonPatch
kubectl	kubectl patch --type=json|merge

COMMON MISTAKES

- Sending a Merge Patch to change one array element — the whole array is replaced with what you sent
- Forgetting that ops run in order — after `add /tags/0`, every later index into `tags` has shifted by one
- Using `null` in a Merge Patch to mean "set to null" — the key is deleted instead; use JSON Patch
- Building pointers by string concatenation without escaping `/` and `~` in keys

TIPS

- Start a JSON Patch with a `test` op on a version field to avoid overwriting someone else's change
- Return `415 Unsupported Media Type` when the `Content-Type` is not the patch format you accept
- Generate patches with `compare(before, after)` rather than writing ops by hand
- `kubectl patch --type=merge` and `--type=json` use these exact formats

SUMMARY

- `JSON Pointer` — `/a/b/0` paths; `-` appends; `~1` = `/`
- `JSON Patch` — ordered ops; atomic; arrays by index
- `test` — guard op; whole patch fails on mismatch
- `Merge Patch` — partial doc; `null` deletes; arrays replaced

Gotchas & Interop

Number.MAX_SAFE_INTEGER · BigInt · dates · NaN · undefined · duplicate keys · BOM · key order

WHERE JSON BITES

JSON is tiny — six types and no options — so every language has to **map** its own types onto it, and they don't all agree. Large integers lose precision in JavaScript, there is **no date type**, NaN and `Infinity` are not valid JSON, and the spec leaves **duplicate keys** and **key order** undefined. Most production JSON bugs come from these gaps, not from syntax errors.

THE RULES TO REMEMBER

- JSON numbers have no size limit, but JavaScript stores them as 64-bit floats: integers above `2^53 - 1` lose precision
- Dates travel as strings — use ISO 8601 / RFC 3339 with an offset: `"2026-10-23T14:05:00Z"`
- Valid JSON must be UTF-8 (RFC 8259) and must not start with a BOM
- Objects are unordered — never rely on key order for meaning or signatures
- Duplicate keys "SHOULD" be unique; most parsers silently keep the last one

JAVASCRIPT

```
JSON.parse('{"id": 9007199254740993}').id  // 9007199254740992  (precision lost)
JSON.stringify({ n: 10n })                 // TypeError: Do not know how to serialize a BigInt
JSON.stringify({ d: new Date(0) })         // '{"d":"1970-01-01T00:00:00.000Z"}'
JSON.parse('{"d":"1970-01-01T00:00:00.000Z"}').d   // a string, not a Date
JSON.stringify({ x: NaN, y: Infinity })    // '{"x":null,"y":null}'
JSON.stringify({ u: undefined, f() {} })   // '{}'   (keys dropped)
JSON.stringify([undefined, () => 1])       // '[null,null]'
JSON.parse('{"a": 1, "a": 2}')             // { a: 2 }  (last wins)
JSON.parse('\uFEFF{}')                     // SyntaxError (BOM)
JSON.stringify({ b: 1, a: 2, 10: "x" })    // '{"10":"x","b":1,"a":2}'  (integer keys first)
```

PYTHON

```
json.loads("9007199254740993")        # 9007199254740993  (int is exact)
json.loads('{"x": NaN}')              # {'x': nan}   — accepts non-standard JSON
json.dumps(float("nan"))              # 'NaN'        — emits invalid JSON!
json.dumps(float("nan"), allow_nan=False)  # ValueError
json.dumps({1: "a", None: "b"})       # '{"1": "a", "null": "b"}'  (keys → strings)
json.dumps(datetime.now())            # TypeError: not JSON serializable
json.dumps(obj, default=str)          # fallback: str() anything unknown
json.loads("0.1", parse_float=Decimal)   # Decimal('0.1')  (exact money)
```

NOTES

- → `9007199254740993` — `Number.MAX_SAFE_INTEGER + 2`; JS rounds to the nearest representable float
- → `10n` — `BigInt` has no JSON form; convert with `.toString()` or add a `toJSON` method
- → `Date` — serialized through its `toJSON()` (ISO string); parsing never revives it
- → `NaN` / `Infinity` — become `null` in JS; Python writes them raw unless `allow_nan=False`
- → `undefined` / functions — dropped from objects, `null` inside arrays
- → `\uFEFF` — byte-order mark; strip it before parsing files saved by some Windows editors
- → Integer-like keys — JS objects list them first in ascending order, whatever order you wrote
- → `default=str` — handles dates, UUIDs, Decimals on the way out

PROBLEM → FIX

PROBLEM	FIX
IDs > 2^53	send as strings: `"id": "9007199254740993"`
money	integer cents or decimal strings, never floats
dates	ISO 8601 with Z or offset; parse explicitly
NaN / Infinity	`null` or a string sentinel; `allow_nan=False`
duplicate keys	reject in parser (`object_pairs_hook`) or lint
key order	canonicalize (RFC 8785 JCS) before hashing/signing
BOM / encoding	read as UTF-8; Python `encoding="utf-8-sig"`

BIG NUMBERS BY LANGUAGE

LANGUAGE	DEFAULT FOR JSON NUMBERS	EXACT OPTION
JavaScript	float64	`BigInt` + custom parser (`json-bigint`)
Python	`int` exact, `float` for decimals	`parse_float=Decimal`
Go	`float64` into `any`	`decoder.UseNumber()` → `json.Number`
Java (Jackson)	`int` / `long` / `double`	`USE_BIG_DECIMAL_FOR_FLOATS`
C# (System.Text.Json)	type of target property	`decimal`, `long`
PostgreSQL	`jsonb` numeric (exact)	`jsonb` drops duplicate keys, reorders

COMMON MISTAKES

- Returning database IDs as JSON numbers — a JS client silently rounds `9007199254740993` and fetches the wrong record
- Storing prices as floats — `0.1 + 0.2` problems reach invoices; use integer cents or decimal strings
- Assuming `JSON.parse` revives dates — it returns strings; compare or sort them as dates only after converting
- Signing or hashing `JSON.stringify(obj)` — different key order or whitespace on another system breaks the signature

TIPS

- Add `"id_str"`-style string IDs (as Twitter/X did) when clients include JavaScript
- Use a reviver to revive dates: `JSON.parse(s, (k, v) => k.endsWith("At") ? new Date(v) : v)`
- Python: `json.loads(s, object_pairs_hook=...)` can raise on duplicate keys
- Document units and time zones in the field name or schema: `durationMs`, `createdAt` (UTC)

SUMMARY

- `2^53 - 1` — largest safe integer in JS; send bigger IDs as strings
- `Date` — no JSON type; ISO 8601 strings both ways
- `NaN` / `undefined` — not JSON; become `null` or vanish
- `duplicate keys` / `order` — undefined; never depend on them

Sheet 14

Quick Reference — Syntax & Schema

syntax rules · value types · escapes · JSON Schema types · object & array keywords · composition · formats · common errors

SYNTAX RULES

RULE	DETAIL	#
`{ "k": v }`	object; keys are double-quoted strings	01
`[ v, v ]`	array; ordered values	01
`, between items`	no trailing comma	01
`"double quotes"`	single quotes invalid	01
`no comments`	use JSONC / JSON5 if needed	08
`UTF-8`	required for interchange (RFC 8259)	01

VALUE TYPES

TYPE	EXAMPLE	#
`string`	`"text"`	02
`number`	42, `-3.5`, `1e3` (no NaN/Infinity)	02
`boolean`	`true` / `false`	02
`null`	`null`	02
`object`	`{"a": 1}`	03
`array`	`[1, "x", null]`	03

ESCAPES

ESCAPE	MEANS	#
`\" \\ \/`	quote, backslash, slash	02
`\n \r \t`	newline, return, tab	02
`\b \f`	backspace, form feed	02
`é`	Unicode code unit	02
`😀`	surrogate pair (emoji)	02

SCHEMA BASICS

KEYWORD	DOES	#
`$schema`	dialect, e.g. draft 2020-12	04
`$id / $ref`	identify / reuse a schema	04
`$defs`	local definitions	04
`type`	`"string"`, `["number","null"]`	04
`enum / const`	allowed value(s)	04
`title / description`	documentation	04

OBJECTS & ARRAYS

KEYWORD	DOES	#
`properties`	schema per key	04
`required`	keys that must exist	04
`additionalProperties`	`false` = no extra keys	04
`items`	schema for every element	04
`minItems / uniqueItems`	size, no duplicates	04
`prefixItems`	tuple positions	04

STRINGS & NUMBERS

KEYWORD	DOES	#
`minLength / maxLength`	string length	04
`pattern`	regex the string must match	04
`format`	`email`, `date-time`, `uri`…	04
`minimum / maximum`	inclusive bounds	04
`exclusiveMinimum`	strict bound	04
`multipleOf`	divisible by	04

COMPOSITION

KEYWORD	DOES	#
`allOf`	must match every schema	04
`anyOf`	at least one	04
`oneOf`	exactly one	04
`not`	must not match	04
`if / then / else`	conditional rules	04
`dependentRequired`	key A needs key B	04

COMMON ERRORS

MESSAGE	CAUSE	#
`Unexpected token '`	single quotes	01
`Unexpected token }`	trailing comma	01
`Unexpected end of input`	truncated / unbalanced	05
`Bad control character`	raw newline in a string	02
`Precision loss`	ints above 2^53 in JS	02
`Circular structure`	cyclic object in stringify	05

⚠ COMMON MISTAKES

- Trailing commas, comments or single quotes in strict JSON
- Large IDs as numbers — JavaScript loses precision past `2^53`; send them as strings
- Treating `format` as enforced — many validators only annotate it unless asked to assert

✓ TIPS & BEST PRACTICES

- Validate payloads against a schema at the boundary of your app
- Set `additionalProperties: false` to catch typos in keys
- Use ISO 8601 strings for dates; JSON has no date type

QUICK SUMMARY

- Six types: string, number, boolean, null, object, array
- Keys and strings use double quotes only
- Schema: `type`, `properties`, `required`, `items`
- Combine schemas with `allOf` / `anyOf` / `oneOf`

Sheet 15

Quick Reference — Parsing, Querying & APIs

parse & serialize by language · JSON.stringify options · JSONPath · jq · Pointer & Patch · HTTP & APIs · related formats

PARSE & SERIALIZE

LANGUAGE	PARSE / STRINGIFY	#
JavaScript	`JSON.parse(s)` / `JSON.stringify(v)`	05
Python	`json.loads(s)` / `json.dumps(v)`	05
Java	Jackson `readValue` / `writeValueAsString`	05
C#	`JsonSerializer.Deserialize` / `Serialize`	05
Go	`json.Unmarshal` / `json.Marshal`	05
Ruby	`JSON.parse` / `obj.to_json`	05

JAVASCRIPT OPTIONS

CALL	DOES	#
`stringify(v, null, 2)`	pretty-print, 2 spaces	05
`stringify(v, ["a","b"])`	only these keys	05
`stringify(v, (k, v) => …)`	replacer function	05
`parse(s, (k, v) => …)`	reviver (revive dates)	05
`toJSON()`	custom serialization	05
`undefined / functions`	dropped from objects	05

PYTHON OPTIONS

CALL	DOES	#
`dumps(v, indent=2)`	pretty-print	05
`sort_keys=True`	stable key order	05
`ensure_ascii=False`	keep Unicode as-is	05
`default=str`	fallback for dates etc.	05
`json.load(f) / dump(v, f)`	files	05
`separators=(",", ":")`	compact output	05

JSONPATH

EXPRESSION	SELECTS	#
`$`	root	06
`$.store.book`	child by name	06
`$..author`	every author, any depth	06
`$.book[0] / [-1]`	first / last	06
`$.book[*].title`	all titles	06
`$.book[?@.price<10]`	filter (RFC 9535)	06

JQ

FILTER	DOES	#
`.`	pretty-print input	06
`.a.b / .[0]`	path / index	06
`.[]`	iterate an array	06
`map(.name)`	transform each	06
`select(.age > 30)`	filter	06
`-r / -c`	raw strings / compact	06

POINTER & PATCH

SYNTAX	DOES	#
`/users/0/name`	JSON Pointer path	06
`~0 / ~1`	escaped ~ / /	06
`{"op":"add",…}`	JSON Patch operation	07
`replace / remove`	other Patch ops	07
`move / copy / test`	more Patch ops	07
`merge-patch+json`	partial object update	07

HTTP & APIS

ITEM	VALUE	#
`Content-Type`	`application/json`	07
`Accept`	`application/json`	07
`Errors`	`application/problem+json` (RFC 9457)	07
`201 / 204`	created / no content	07
`400 / 422`	malformed / invalid body	07
`res.json()`	fetch: parse response	07

RELATED FORMATS

FORMAT	ADDS	#
JSONC	comments (VS Code configs)	08
JSON5	comments, trailing commas, quotes	08
NDJSON / JSON Lines	one value per line	08
JSON-LD	linked data `@context`	08
BSON / CBOR	binary encodings	08
GeoJSON	geometry objects	08

⚠ COMMON MISTAKES

- `JSON.parse` on untrusted input without try/catch — a bad body throws
- `JSON.stringify` silently dropping `undefined`, functions and `Map` values
- Sending JSON without a `Content-Type: application/json` header

✓ TIPS & BEST PRACTICES

- Stream large files as NDJSON instead of one giant array
- Use `jq` to explore API responses from the command line
- Return `problem+json` error bodies with `type`, `title`, `status`, `detail`

QUICK SUMMARY

- Every language has parse + stringify in its standard library or one package
- JSONPath / jq query; Pointer addresses; Patch edits
- APIs: `application/json` in and out, 4xx for bad bodies
- JSON5 / JSONC for configs; NDJSON for streams